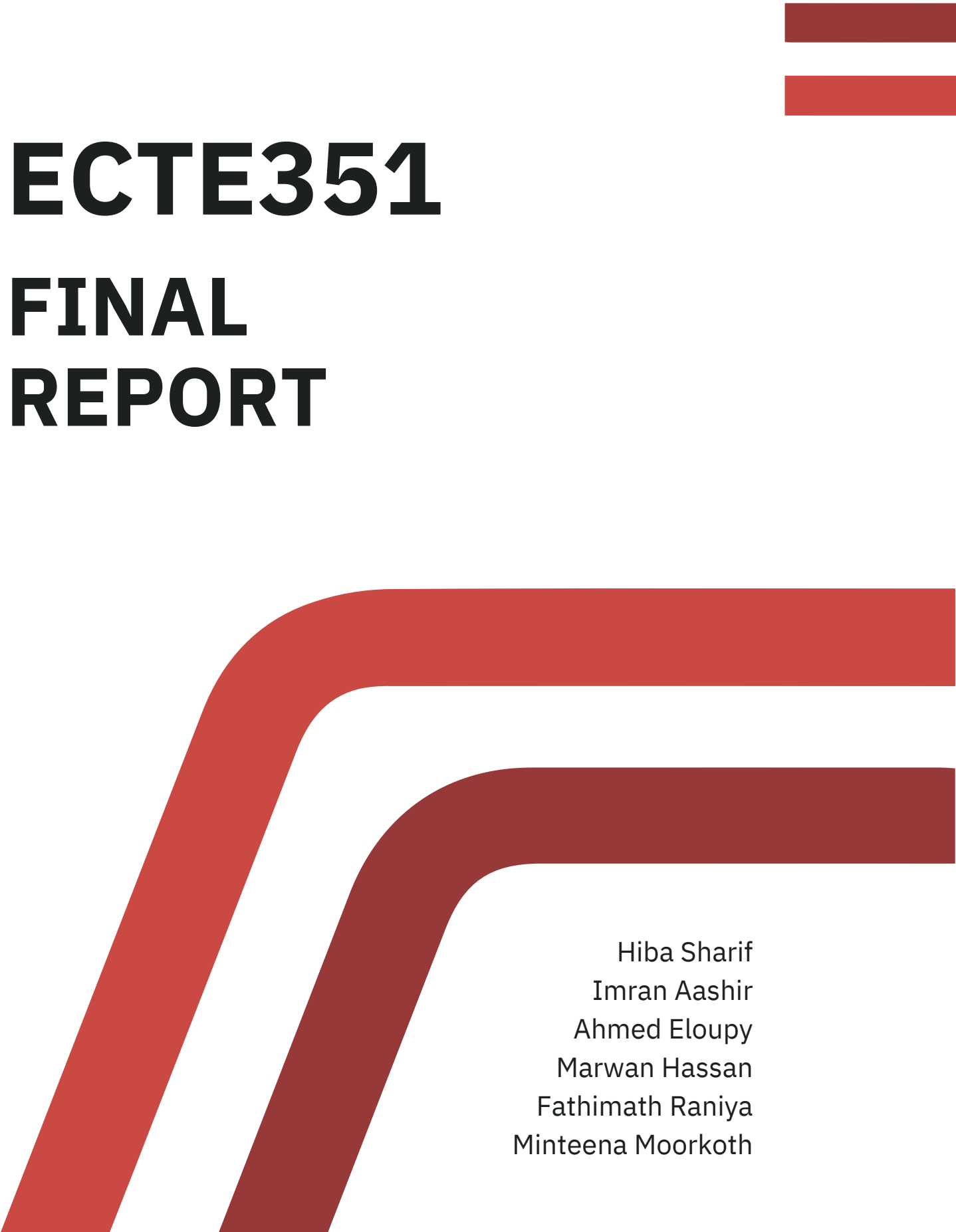


ECTE351

FINAL REPORT



Hiba Sharif
Imran Aashir
Ahmed Eloupy
Marwan Hassan
Fathimath Raniya
Minteena Moorkoth

Table of Contents

1. Executive Summary	3
1.1 Key Project Achievements	3
1.2 Potential Societal and Municipal Impact	3
2. Introduction	4
3. Project Scope	5
3.1 In-Scope Functionality	5
3.2 Out-of-Scope Considerations	5
4. Design Overview	6
4.1 Functional Diagram	6
5. Design Hardware	7
5.1 Block Diagram	7
5.2 Components	8
6. Power Analysis	12
6.1 Component – by – Component Breakdown	12
6.2 Estimated Battery Duration (Min–Max)	13
6.3 Power Optimization Methods	13
7. Software Design & Implementation	14
7.1 ROS 2 Integration	14
7.2 SLAM for Mapping	14
7.3 Image Processing	14
7.4 Data Publishing	15
8. Integration & Testing	15
8.1 Integration of Software Modules	15
8.2 Hardware - ROS Interfacing	16
8.3 Dashboard and Backend Pipeline	16
8.4 Simulation Testing	17
8.5 Pothole Detection Accuracy	17
9. Critical Discussion of Prototype	18
9.1 Objectives and Goals	18
9.2 Methodology Followed	18
9.3 Iteration Breakdown & Team Roles	18
9.4 Critical Discussion	20
10. Challenges & Solutions	20
11. Results and Achievements	22
12. Impact & Value Proposition	22
13. Costing & Financial Model	23

13.1 Development Cost (Prototype Phase).....	23
13.2 Operational/Production Cost (Per Unit).....	24
13.3 Business Model – Data-as-a-Service (DaaS).....	24
13.4 Estimated Yearly Revenue.....	25
13.5 ROI Overview	25
14. Recommendations	25
14.1 Technical Enhancements	26
14.2 Dashboard and Interface Enhancements.....	26
14.3 Deployment Strategies.....	27
14.4 Scalability & Innovation	27
15. Sustainability and SDG Alignment	27
15.1 Reducing Environmental Impact Through Smart Inspection	28
15.2 Social Sustainability and Workforce Safety.....	28
15.2.1 Economic Sustainability Through Preventive Maintenance	28
15.2.2 Alignment with SDG 11: Sustainable Cities and Communities	29
15.2.3 Long-Term Vision for Green Infrastructure Management	29
16. Conclusion & Future Work.....	29
17. References	30
18. Appendix.....	31

Table of Figures

Figure 4.1: Functional Diagram.....	6
Figure 5.1: Block Diagram.....	7
Figure 5.2.1: Turtlebot3 Burger.....	8
Figure 5.2.2: Raspberry Pi.....	8
Figure 5.2.3: Raspberry Pi Work Flow.....	9
Figure 5.2.4: Raspberry Pi Camera.....	9
Figure 5.2.5: LDS – 02.....	10
Figure 5.2.6: LDS - 02 Work Flow.....	10
Figure 5.2.7: Battery.....	10
Figure 5.2.8: Dashboard.....	11
Figure 5.2.9: Robot with the designed enclosure.....	12
Figure 5.2.10: Physical Environment Setup.....	12
Figure 8.2.1: Pothole Detection.....	16
Figure 8.2.2: Robot with Camera attached.....	16
Figure 8.3.1: Custom Dashboard.....	16
Figure 8.4.1: Simulation Testing.....	17
Figure 9.3.1: Initial Environment.....	18
Figure 9.3.2: Modified Environment.....	19
Figure 9.3.3: Markers Display.....	19
Figure 14.2.1: Dashboard (Home Page).....	26
Figure 14.2.2: Dashboard (Analytics Page).....	27
Figure 18.1: Gantt Chart.....	31
Figure 18.2: PaveWay Poster.....	32
Figure 18.3: Track Measurements.....	32

1. Executive Summary

Urban infrastructure maintenance is a cornerstone of sustainable development and public safety. As cities expand, the demand for efficient and intelligent inspection systems is increasing. Traditional manual inspection methods are often inefficient, inconsistent, and risky. **PaveWay, An Autonomous Road Inspection Robot**, developed by our team *InfraSense*, addresses these challenges by offering a robust, automated alternative that leverages modern robotics, AI, and geospatial technologies.

PaveWay integrates SLAM, image-based defect detection, and geotagging to inspect roads without direct human supervision. Built on the TurtleBot3 platform and powered by a Raspberry Pi, the robot features LiDAR, a Raspberry Pi camera, motor drivers, and Wi-Fi connectivity. Together, these systems enable autonomous navigation, real-time defect logging, and centralized dashboard monitoring for authorities responsible for road infrastructure.

1.1 Key Project Achievements

The robot's design and implementation have led to several significant accomplishments:

- **Autonomous Navigation:** Achieved real-time, obstacle-aware SLAM-based movement in testing environments.
- **Defect Recognition:** Integrated AI algorithms for pothole detection using visual data and pattern recognition.
- **Data Management:** Implemented real-time geotagging and data logging capabilities for streamlined reporting.
- **Dashboard Integration:** Enabled live monitoring through a user-friendly dashboard interface.
- **Collaborative Development:** Successfully coordinated mechanical design, electronics integration, software development, and testing within a team of six engineers.

1.2 Potential Societal and Municipal Impact

The adoption of PaveWay by municipalities could drastically improve road condition monitoring efficiency. It enables continuous inspection routines, reducing reliance on human personnel and improving response times to hazards. Ultimately, this translates to:

- **Lower maintenance costs** through early defect detection.
- **Increased road safety**, minimizing vehicle damage and personal injury.
- **Smarter urban planning** through data-driven infrastructure analysis.

In conclusion, PaveWay aligns strongly with global priorities for urban sustainability, supporting proactive infrastructure maintenance and fulfilling core objectives under Smart City development frameworks.

2. Introduction

Modern urban mobility depends heavily on the quality of roads and public thoroughfares. As cities in the UAE and around the world experience rapid growth, their road infrastructure is subjected to increasing wear from traffic, climate conditions, and material degradation. Maintenance of these surfaces has become a pressing concern—yet, current inspection practices are outdated, inconsistent, and heavily reliant on manual labor.

Most municipalities rely on human inspectors who must visually assess road conditions, a process that is:

- Time-consuming and labor-intensive
- Expensive and operationally inefficient
- Subjective in judgment
- Unsafe due to exposure to traffic and harsh environments

This reactive model often fails to catch emerging defects early, leading to escalating repair costs and higher risks to public safety. For instance, studies show that potholes and undetected cracks contribute significantly to vehicle damage and accidents—costing billions annually worldwide in avoidable expenses.

Despite advances in mapping and sensing technologies, few cities have adopted intelligent, ground-level inspection tools. Drones and satellite imagery lack the granularity needed to detect fine surface anomalies. A smart, real-time, automated solution is needed to fill this gap.

PaveWay: A Smart Inspection Solution for Road Infrastructure

To address these challenges, the InfraSense team developed PaveWay, a compact, autonomous robot designed to inspect roads using a combination of:

- AI-powered visual defect detection
- SLAM and LiDAR-based navigation
- Geotagged data logging
- Centralized dashboard reporting for infrastructure authorities

The robot's modular design, affordability, and adaptability make it ideal for integration into government or private inspection frameworks. By shifting from reactive to proactive maintenance, PaveWay helps ensure safer roads, smarter resource allocation, and longer infrastructure lifespans.

3. Project Scope

The PaveWay project was scoped with a focus on road inspection, specifically, detecting and reporting surface anomalies using an autonomous robot. The system was designed for city authorities and infrastructure service providers to monitor road health via a live dashboard interface.

3.1 In-Scope Functionality

The following capabilities were designed, developed, and integrated within the project framework:

- **Pavement Defect Detection**
Using AI-based image processing techniques, the system identifies potholes, cracks, and surface anomalies with reasonable accuracy under varying lighting conditions.
- **Real-Time Geotagging**
Detected anomalies are linked to GPS coordinates (via connected modules), enabling city maintenance teams to pinpoint defect locations for targeted action.
- **Data Logging and Visualization**
A structured log of inspections is maintained, including timestamps, locations, and image samples. These are visualized on a centralized live dashboard.
- **Autonomous Navigation**
Utilizing LiDAR and SLAM algorithms, the robot autonomously maps its surroundings and navigates pavements while avoiding obstacles.
- **Live Dashboard Integration**
A responsive dashboard provides access to real-time insights, enabling quick decision-making for repair scheduling and resource allocation.

3.2 Out-of-Scope Considerations

To maintain focus and feasibility within the project timeline, certain features were excluded from this version:

- **Physical Repair Capabilities:** The system does not include tools or attachments for pothole filling or crack sealing.
- **Multi-Robot Coordination:** Fleet coordination protocols and cloud-based multi-agent systems are outside the scope of this prototype.

4. Design Overview

The PaveWay Autonomous Pavement and Road Inspection Robot was developed to provide a scalable, low-cost solution for detecting and reporting road surface defects on both pedestrian pavements and vehicular roads. The system integrates sensing, mapping, and data communication technologies within a compact autonomous robot to automate the traditionally labor-intensive process of road infrastructure monitoring.

The robot is built on the TurtleBot3 Burger platform, selected for its native ROS 2 compatibility, integrated LiDAR, and compact design. Its core functionality hinges on the synergy between LiDAR-based SLAM for mapping and navigation, camera-based image processing for pothole detection, and real-time wireless data communication for reporting.

The physical environment for testing was developed in parallel with the robot. It involved creating accurate artificial potholes and surface textures using boards carved to resemble pavement damage. This played a crucial role in validating both the vision and navigation systems under realistic constraints.

The final design allowed PaveWay to complete autonomous navigation and image-based pothole detection in a realistic test setting, demonstrating not only the feasibility of the approach but also its potential for future deployment in urban environments.

4.1 Functional Diagram

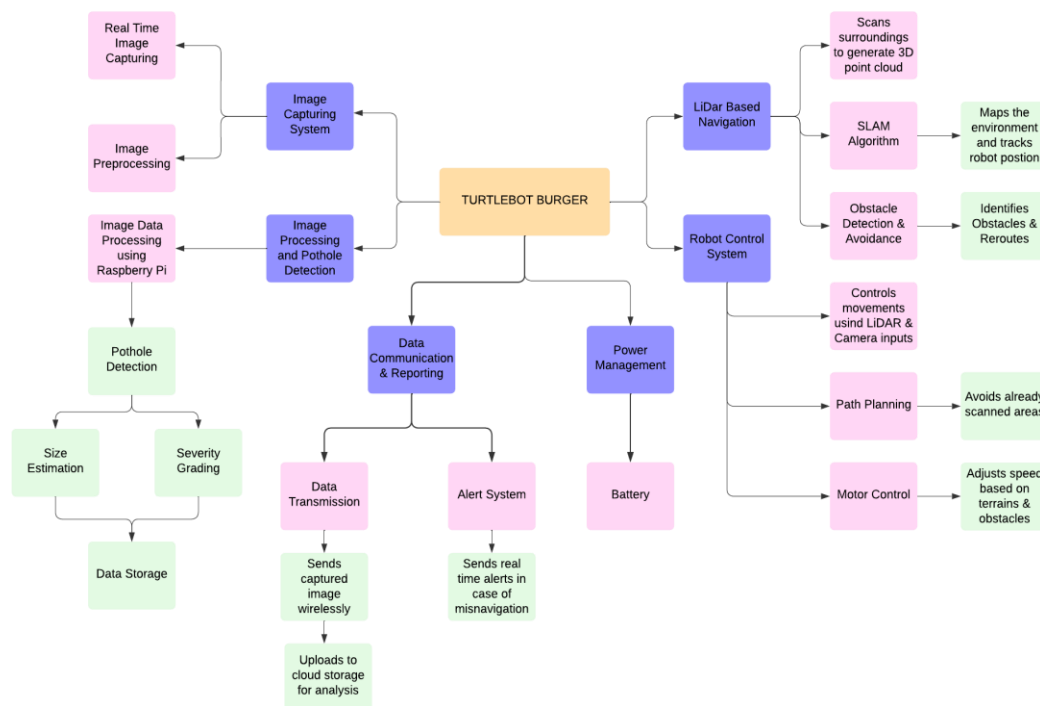


Figure 4.1: Functional Diagram

The above functional diagram showcases the processes involved in the operation of the robot:

1. **Image Capturing System:** Captures the images of the pavement in real time and preprocesses it before analysis. Using Raspberry Pi it processes the image data to check for potholes.
2. **Pothole Detection:** Detects the potholes through image analysis and grades the severity. This data is stored for future analysis and reporting.
3. **LiDAR Based Navigation:** Scans the surrounding environment to generate a 3D point cloud. Utilizes SLAM to map the surroundings, tracks the robot's position and avoids obstacles, rerouting as necessary.
4. **Robot Control System:** Controls the movements of the robots based on the inputs from the LiDAR and the camera. Plans paths to avoid already scanned areas and adjusts speed according to the terrain.
5. **Data Communication & Reporting:** Uploads data (captured images) to the cloud storage for analysis. Sends alerts to the authorities if the robot mis navigates or encounters any issues.

5. Design Hardware

5.1 Block Diagram

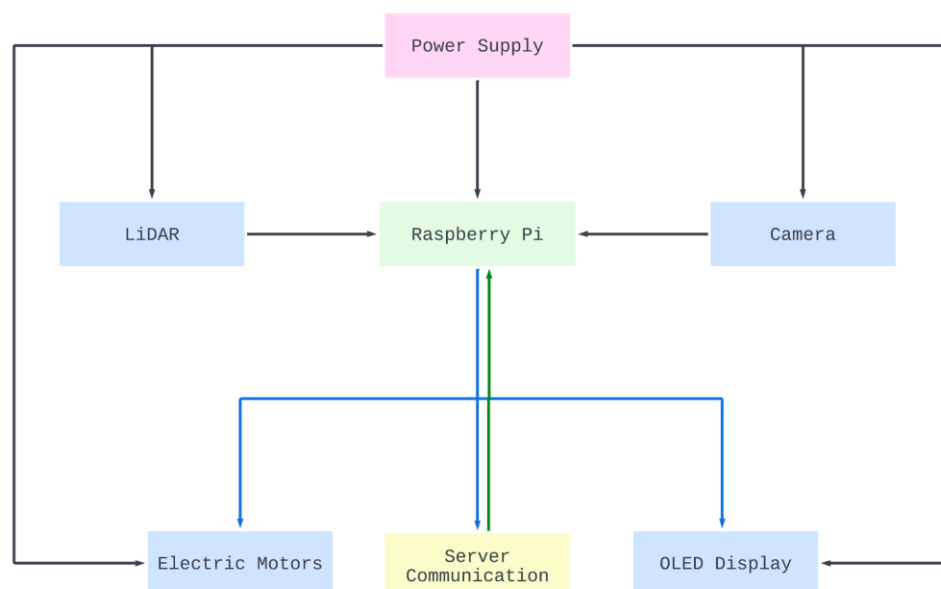


Figure 5.1: Block Diagram

The block diagram of the PaveWay Autonomous Pavement Inspection Robot illustrates the interconnected hardware components that enable its autonomous functionality. Each component plays a critical role: the Power Supply ensures consistent power; the Raspberry Pi serves as the central processor, handling data and sending commands; the Camera captures pavement images and allows for simple navigation; and the LiDAR Sensor allows for greater depth of pothole mapping and pathfinding. Together, these components allow the robot to inspect pavements autonomously, navigate safely, and report defect data for maintenance purposes.

5.2 Components

Platform: TurtleBot3 Burger

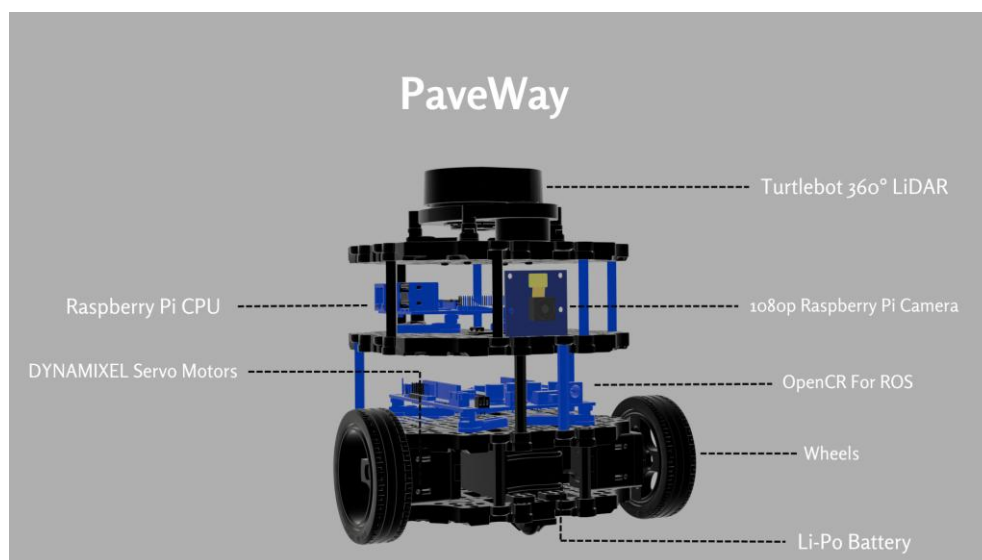


Figure 5.2.1: Turtlebot3 Burger

The TurtleBot3 Burger was chosen as the base platform for its integrated LDS-02 LiDAR sensor, compact footprint, and native ROS 2 compatibility. The modular chassis also provided mounting flexibility for custom-designed enclosures and additional components.

Processing Unit: Raspberry Pi 4

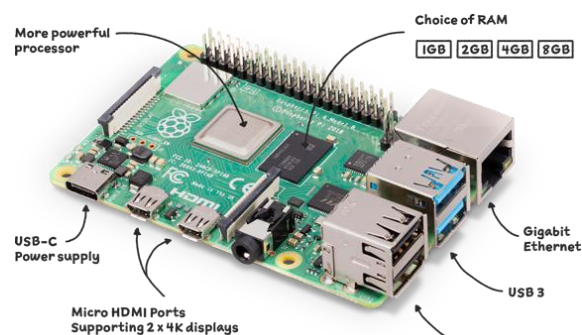


Figure 5.2.2: Raspberry Pi

The Raspberry Pi 4 served as the robot's central computing unit, managing data acquisition, sensor fusion, image processing, and communication. Its USB and GPIO connectivity allowed straightforward interfacing with external modules, and its support for Python and ROS 2 made it ideal for running the full robotics stack, including SLAM, YOLO-based detection, and wireless dashboard updates.

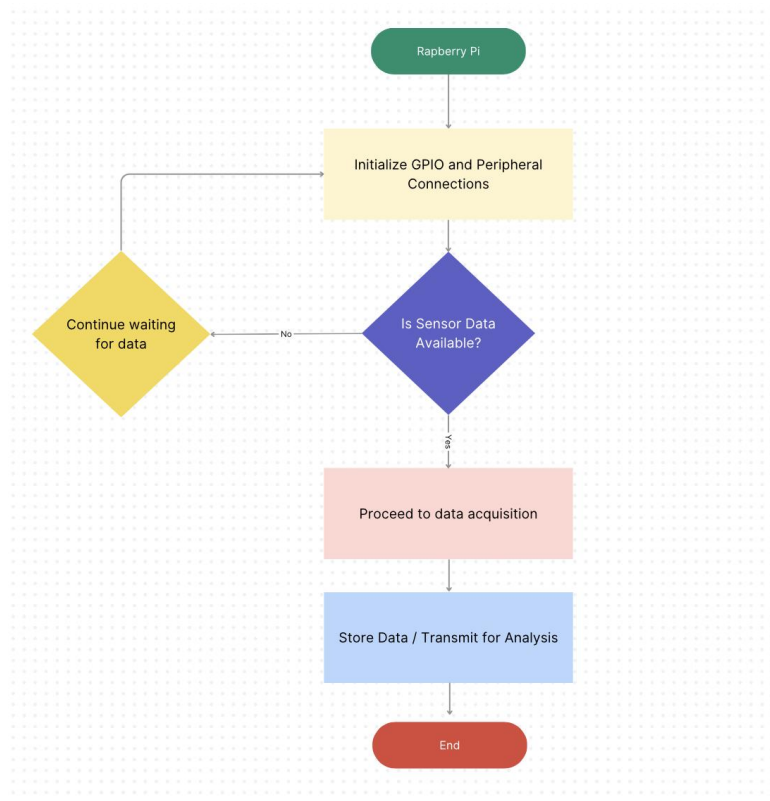


Figure 5.2.3: Raspberry Pi Work Flow

Sensing and Detection Suite:

- **Camera (1080p HD):** A high-definition Raspberry Pi-compatible camera captured live images of the road or pavement surface. It was mounted using a custom 3D-printed bracket, designed and fitted to minimize vibration and optimize the viewing angle for pothole detection.



Figure 5.2.4: Raspberry Pi Camera

- **LiDAR Sensor (LDS-02):** Provided a 360° view of the robot's surroundings, crucial for SLAM-based mapping, obstacle detection, and localization.



Figure 5.2.5: LDS - 02

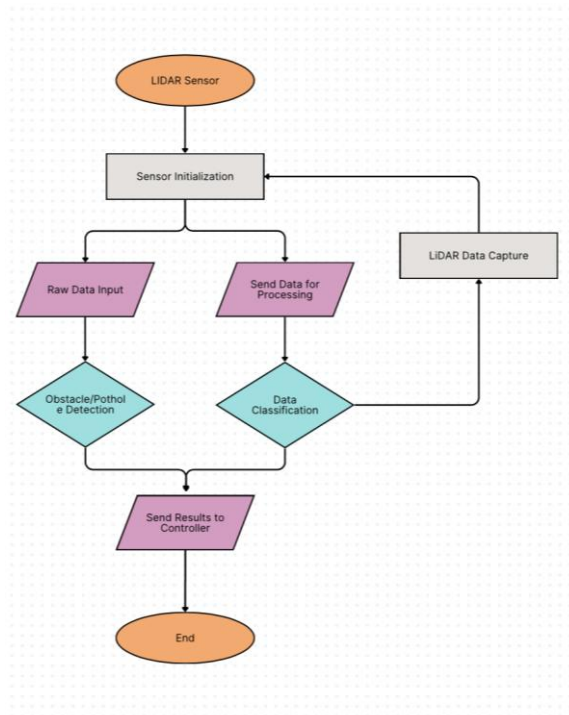


Figure 5.2.6: LDS - 02 Work Flow

Power Supply:



For ROBOTIS OP

Figure 5.2.7: Battery

An 11.1V 1800mAh LiPo battery powered the system, offering approximately 40 minutes of runtime under typical operating conditions.

DC Motor:

PaveWay uses the TurtleBot3's built-in DC motors with encoders to achieve differential drive navigation. These motors are controlled via an onboard motor driver, which receives velocity commands from the Raspberry Pi through ROS 2.

The encoders provide real-time odometry data, essential for accurate localization and SLAM. The motors enable smooth forward, reverse, and turning motion, which was calibrated for slow, stable movement to ensure high detection accuracy during inspection.

Server Communication:

To close the feedback loop between robot and user, the Raspberry Pi communicated with a **web dashboard** hosted on a remote server. Using `rosbridge_server` and **WebSocket protocols**, the robot published real-time data including:

- Pothole detections with severity and location
- Robot pose and live SLAM map
- MJPEG video stream from the onboard camera
- Alerts or diagnostics (e.g., sensor errors, low battery)

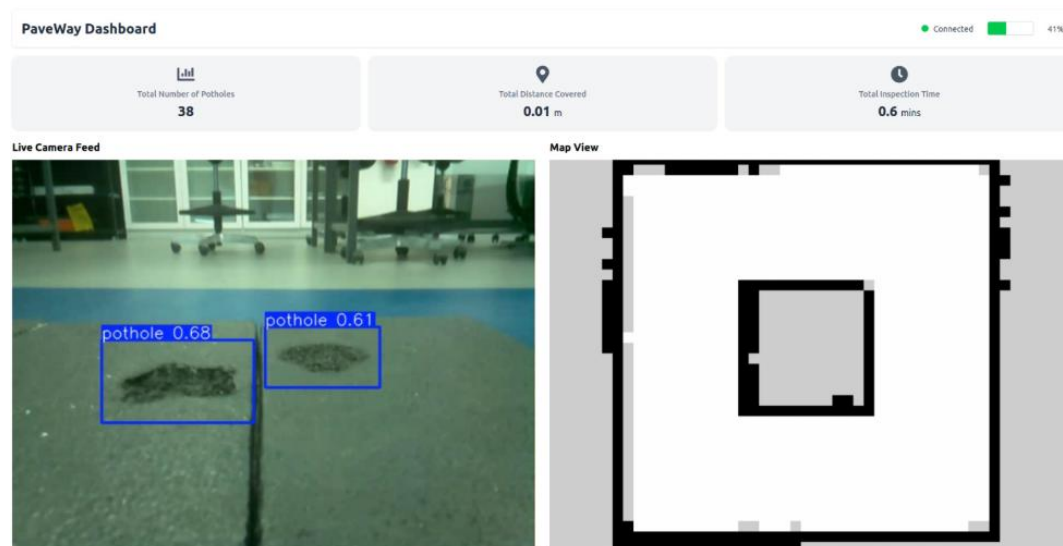


Figure 5.2.8: Dashboard

Custom Mounts and Testing Environment

To house and secure the components, we designed and fabricated custom mounts and enclosures using Fusion 360. These were 3D printed and laser cut to match the TurtleBot's compact geometry while protecting sensors and ensuring proper alignment.



Figure 5.2.9: Robot with the designed enclosure

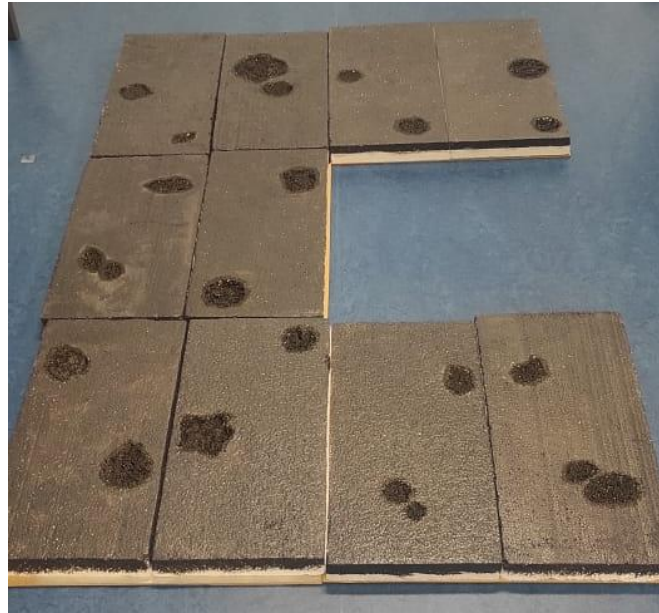


Figure 5.2.10: Physical Environment Setup

6. Power Analysis

6.1 Component – by – Component Breakdown

Component	Voltage (V)	Current (A)	Power (W)	Notes
TurtleBot3 Base (Burger)	12.0	~1.5	18.0	Motors, MCU, IMU, encoders
LDS-02 LiDAR	5.0	0.45	2.25	Scanning LiDAR
Raspberry Pi 4 (4GB)	5.1	2.5	12.75	Onboard computation
Raspberry Pi Camera 3	5.0	0.25	1.25	For pothole image capture
Wi-Fi Module (integrated)	5.0	0.15	0.75	For dashboard communication
Miscellaneous (LEDs, losses)	—	—	~1.0	Power regulators, sensors, etc.

Total Average Power Consumption $\approx$ 36 W

6.2 Estimated Battery Duration (Min–Max)

- **Battery:** TurtleBot3 comes with a **Li-ion 11.1V, 1800 mAh (19.98 Wh)** battery.
- **Realistic System Power Draw:** 30–36 W (Under full load - motors running throughout).
In real world usage, motors are not always kept active.

Considering that, the robot moves approximately **30% of the time and** remains idle or scanning for the remaining **70%**,

The **average system power draw** is estimated to be **~15.3 W**.

Runtime Estimation:

- **Minimum Duration (full load, ~36 W):**

$$\frac{19.98 \text{ Wh}}{36 \text{ W}} \approx 0.55 \text{ hrs} \approx 33 \text{ minutes}$$
- **Maximum Duration (idle-only, ~9 W):**

$$\frac{19.98 \text{ Wh}}{9 \text{ W}} \approx 2.2 \text{ hrs} \approx 132 \text{ minutes}$$
- **Estimated Realistic Runtime (average ~15.3 W):**

$$\frac{19.98 \text{ Wh}}{15.3 \text{ W}} \approx 1.3 \text{ hrs} \approx 78 \text{ minutes}$$

Note: A larger external battery would be required for field deployment.

6.3 Power Optimization Methods

Method	Description
Sleep/Idle Modes	When robot is stationary or paused between missions, lower CPU frequency or suspend non-essential nodes.
ROS 2 Node Scheduling	Adjust processing frequency of image recognition nodes to match need (e.g., 2 Hz instead of continuous).
YOLOv8 Inference Throttling	Limit model inference to specific zones or at set intervals.
Hardware Upgrades	Use Jetson Nano or Coral TPU for more efficient AI inference than Raspberry Pi.

7. Software Design & Implementation

The software framework of PaveWay was constructed with modularity and real-time performance in mind, leveraging the ROS 2 (Robot Operating System 2) middleware to enable flexible communication across distributed computing nodes. The design aimed to ensure that the robot could autonomously map its environment, detect and record potholes, and communicate findings to a remote dashboard in real-time.

7.1 ROS 2 Integration

ROS 2 was chosen due to its real-time capabilities, active development support, and built-in tools for inter-node communication, message passing, and lifecycle management. Each primary functionality—SLAM, image processing, navigation, and data publishing—was encapsulated within dedicated ROS 2 nodes.

Custom packages were developed in a separate workspace (paveway_ws) to maintain separation from the TurtleBot3's default workspace. Standard messages such as sensor_msgs/Image, geometry_msgs/Pose, and nav_msgs/OccupancyGrid were used to ensure compatibility with the broader ROS ecosystem. Interfaces between nodes were designed to be extensible and reusable, allowing seamless testing and swapping of components.

7.2 SLAM for Mapping

For environmental mapping, the project utilized the `cartographer` package, which supports online pose graph optimization. In the physical robot, data from the LDS-02 LiDAR and odometry from the TurtleBot3 base were published to `/scan` and `/odom`, respectively. These were consumed by Cartographer to generate a map incrementally in real-time.

In simulation, Gazebo was configured to provide equivalent `/scan` and `/odom` data. The SLAM map was published on `/map`, and pose estimation on `/pose`. The resultant map was later used by the navigation stack to plan paths and avoid mapped obstacles.

7.3 Image Processing

The image processing subsystem was split into two modes: HSV-based color filtering for simulations and a trained YOLOv8 model for real-world inference. Both approaches relied on live images from either Gazebo's virtual camera or the Raspberry Pi Camera 3.

For HSV filtering, OpenCV was used to apply masking based on calibrated HSV ranges for red potholes. Contours were extracted to localize potholes, and centroid coordinates were computed and transformed into the robot's frame.

The YOLOv8 model, trained on a custom dataset of annotated pothole images, was run directly from a `.pt` PyTorch model file via a Python-based ROS 2 node. Detected potholes were identified by bounding boxes, and their relative position with respect to the robot was estimated by calculating the centroid of each bounding box.

7.4 Data Publishing

To enable real-time visualization and reporting, detected pothole data, robot pose, and map snapshots were published via the `rosbridge_server` protocol to a React-based web dashboard over Wi-Fi. The communication leveraged WebSocket subscriptions to ROS topics.

Data included JSON objects containing timestamped pothole detections, robot location (`x`, `y`, `theta`), and a low-latency MJPEG camera stream. This enabled remote users to view the robot's state and progress via a dynamic, browser-accessible GUI. Authentication and data persistence were not prioritized for the MVP but were documented for future versions.

8. Integration & Testing

Testing and integration were carried out iteratively and in parallel across both simulation and real-world platforms to ensure stability, coherence between modules, and operational readiness.

The integration strategy was designed to test isolated functionalities first before incrementally combining them into a full-stack autonomous system.

8.1 Integration of Software Modules

A modular integration plan was followed, starting with individual validation of each software component—SLAM, camera capture, image processing, and navigation. Next, two-subsystem integrations were tested:

- **Camera + Pothole Detection:** Ensured images were captured, processed, and detections logged correctly.
- **SLAM + Navigation:** Verified that the robot could localize itself and plan paths in both known and partially mapped environments.
- **Detection + Server Publishing:** Validated real-time transmission of pothole data to the web interface.

Finally, all components were launched via unified launch files to allow streamlined startup of the entire stack.

8.2 Hardware - ROS Interfacing

Hardware communication was achieved via the ROS 2 bringup packages on the Raspberry Pi 4. The camera node published to `/camera/image_raw`, the LiDAR to `/scan`, and wheel encoders to `/odom`. Serial and USB interfaces were validated using `dmesg` and `udev` rules.

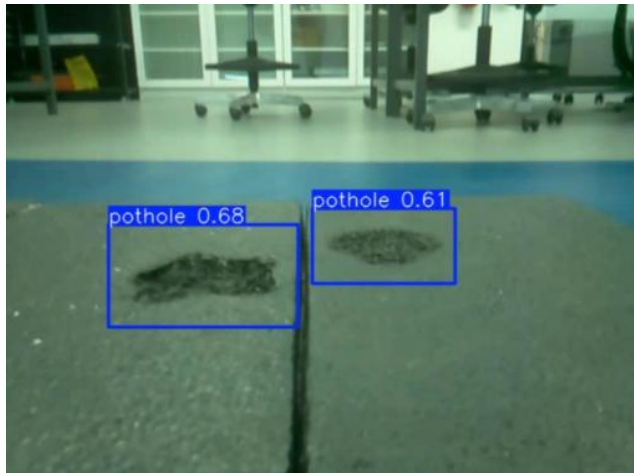


Figure 8.2.1: Pothole Detection

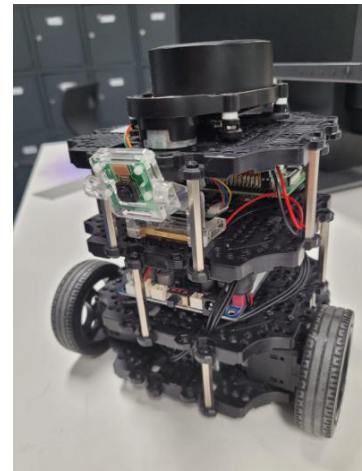


Figure 8.2.2: Robot with Camera attached

8.3 Dashboard and Backend Pipeline

The web dashboard was implemented using React, interfacing with ROS topics via `rosbridge_server`. It received image frames and JSON data through WebSocket subscriptions. The dashboard displayed the robot's current pose, map, live video feed, the total inspection time, no. of potholes detected and the battery level.. Backend testing focused on ensuring data consistency, minimal latency, and fault tolerance. Mock data was used to simulate robot outputs before full pipeline integration.

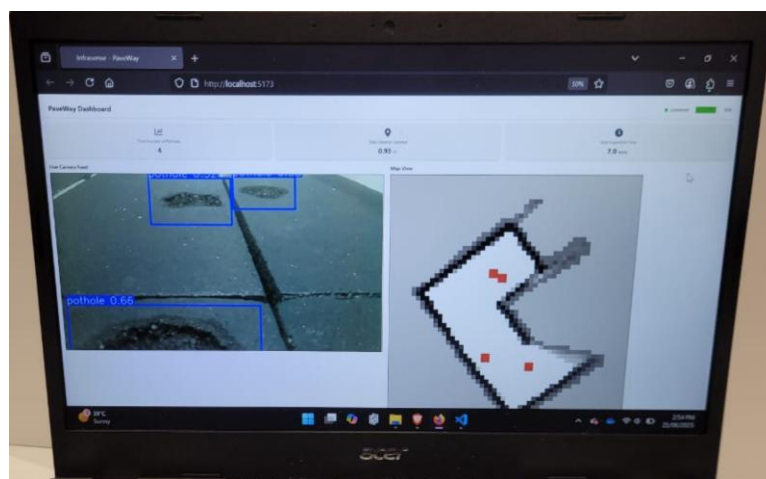


Figure 8.3.1: Custom Dashboard

8.4 Simulation Testing

Simulation environments in Gazebo and RViz2 were critical for testing without hardware dependencies. A test world with artificial red patches representing potholes was created. The robot was driven using Nav2 through this world while the HSV-based detection node identified red spots.

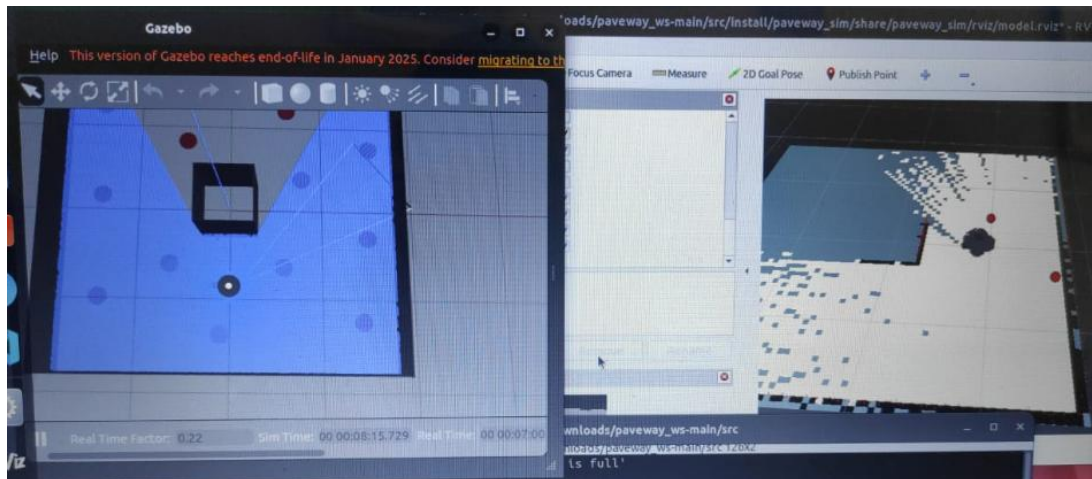


Figure 8.4.1: Simulation Testing

RViz2 was used to monitor SLAM maps, robot pose, and published goals. It also allowed for rapid debugging of transform frames (`tf2`) and data topics.

8.5 Pothole Detection Accuracy

Detection performance was benchmarked against a labeled dataset of pothole images in diverse lighting and texture conditions. The YOLOv8 model achieved a mean Average Precision (mAP) of 88.4% on the test set. False positives were mainly due to shadows or similarly colored objects.

Physical testing was conducted in an indoor environment configured with realistic pothole representations. Coordinates were compared in the global map frame to assess positional accuracy, with errors typically under 15 cm.

9. Critical Discussion of Prototype

9.1 Objectives and Goals

The design iteration process in this project was crucial for refining both the hardware and software systems. The goal was to build a fully functional, autonomous road inspection robot capable of real time road defect detection, environmental mapping and live data display on the dashboard. Each iteration targeted specific aspects of the system, such as navigation accuracy, defect recognition, fitting of the components and data integration.

9.2 Methodology Followed

Development was broken into functional modules: Mapping & Navigation, Detection and Communication. Each module was developed, tested both in simulation and in physical setup, reviewed based on observed limitations, and improved in the next cycle. Each team member was assigned responsibility for leading on or more modules and providing feedback during team reviews.

9.3 Iteration Breakdown & Team Roles

Iteration 1: Real Environment Setup

The first major iteration involved the development of a realistic test environment. Initially, we decided to use wooden boards and modelling clay to simulate roads and potholes, which was done by Fathimath Raniya and Imran Aashir. While this allowed for quick prototyping, the lack of visual realism and texture made camera detection unreliable. This led us to consult a media professor, who suggested carved styrofoam boards, which were later painted black to resemble asphalt. The change significantly improved both the appearance and functional testing accuracy. All team members contributed to this redesigned environment.



Figure 9.3.1: Initial Environment



Figure 9.3.2: Modified Environment

Iteration 2: Visual Detection & Image Processing

Another crucial iteration centred around the development and testing of the pothole detection model. Using OpenCV on a Raspberry Pi camera, our teammate, Hiba Anwar tested various thresholding and contour filtering methods to ensure that only true defects were captured. Early versions of the model misidentified shadows and floor textures as potholes, especially under inconsistent lighting. Through extensive testing and tuning of parameters, she refined the model to reliably detect surface defects within the physical environment.

Iteration 3: RViz Marker Display

During simulation, Minteena Moorkoth used the Turtlebot3 Waffle Pi model, which already included camera frames in its URDF, allowing it to be displayed correctly. However, transitioning to the Turtlebot3 Burger for real life deployment required her to manually add and configure the camera frames in the URDF file. This caused significant mismatch in the marker locations, with pothole markers appearing offset on the map. She had to carefully adjust the frame transforms and tune the threshold values that determine when a pothole should be geotagged. This process was time consuming and required multiple rounds of testing.

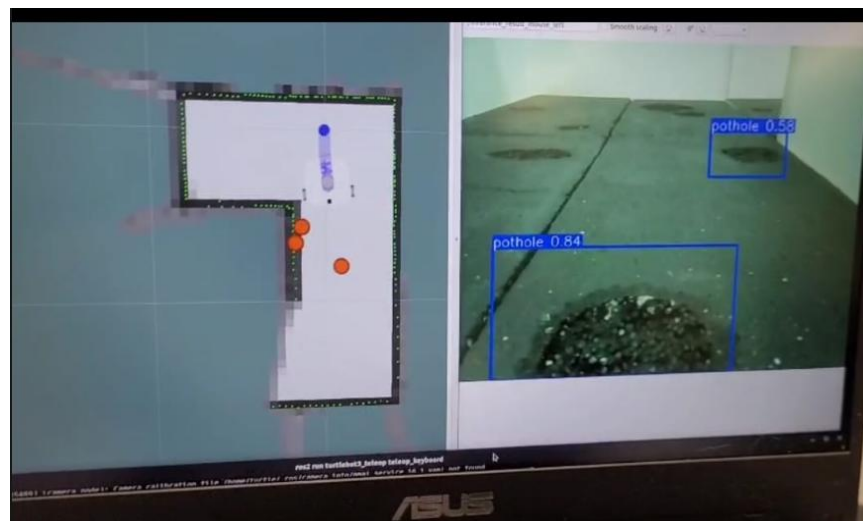


Figure 9.3.3: Markers Display

9.4 Critical Discussion

Our iterative process helped us transform a basic idea into a function prototype. Early stages focused on feasibility, but each cycle improved technical depth, and real world applicability. By isolating the different modules (mapping & navigation, detection, communication) we could troubleshoot issues faster and integrate only when these subsystems were stable.

Each of these iterations enhanced the overall system and reinforced the importance of adaptive design. Although the 3D camera holder required redesign due to fitting issues, which was done by Imran Aashir, it was considered a minor adjustment compared to the functional challenges of integrating data and ensuring proper visualization.

Team roles evolved naturally. Some initially assigned tasks were shifted due to availability of time or when team members showed better strengths in completing other tasks. We overcame technical challenges through documentation, mentorship (e.g. consulting the media professor), and through peer review. The use of simulation (Gazebo) fastened our software iteration, while the real world test environment grounded our results.

Without these design iterations, the system wouldn't have reached its current level of performance and presentation readiness. These cycles allowed us to identify places of errors, correct these errors and ultimately deliver a working demo aligned with the original project goals.

10. Challenges & Solutions

Challenge 1: Achieving Reliable Autonomous Navigation for Road Scenarios

Problem: Deploying the robot directly onto real roads was impractical due to TurtleBot's size and limitations.

Solution: To overcome this, the team constructed a miniature simulated road environment, replicating realistic road features, markings, and potholes at scale. This allowed them to develop and validate their navigation and mapping algorithms under controlled yet realistic conditions, ensuring the robot's SLAM-based localization and obstacle avoidance logic were prepared for potential future deployment with upgraded hardware.

Challenge 2: Accurate Pothole Detection under Varying Lighting Conditions

Problem: Even within the simulated environment, lighting variations such as shadows, glare from overhead lights, and material reflectivity posed challenges to reliable pothole detection accuracy.

Solution: The team implemented a machine learning-based image processing approach with adaptive thresholding and filtering calibrated against these lighting inconsistencies. Testing in the miniature setup under controlled lighting simulations improved detection precision and allowed severity classification models to be validated effectively.

Challenge 3: Limitations in Battery Life for Extended Inspections

Problem: The ~1.42 hours runtime provided by the 11.1V LiPo battery restricted long-duration testing, posing a limitation even within the simulated environment for continuous operation studies.

Solution: Although battery upgrades were outside current project scope, the team optimized runtime by integrating power distribution management and a battery management system to stabilize output, prevent power dips, and ensure maximum utilization of available charge for effective testing cycles.

Challenge 4: Complex Multi-Sensor and Module Integration

Problem: Integrating LiDAR, camera, GPS modules, motor control, and wireless data transmission into a seamless operational system required overcoming synchronisation, latency, and real-time processing bottlenecks.

Solution: A modular integration strategy was employed, with each subsystem validated independently within the miniature road environment before full integration under ROS. This ensured reliable sensor fusion, precise motor control, and stable real-time data transmission, even during continuous operation tests simulating real road inspections.

Challenge 5: Delivering a Road-Focused Inspection System within Budget Constraints

Problem: Designing a realistic, functional road inspection system within a tight student project budget (~1500 AED) while ensuring relevance for actual road maintenance applications.

Solution: The team strategically leveraged provided TurtleBot and LiDAR units for proof-of-concept development, investing in critical upgrades such as the HD camera, GPS module, and materials for the miniature road simulation. This approach ensured that testing environments realistically reflected road conditions without requiring prohibitively expensive outdoor deployment trials during this development phase.

11. Results and Achievements

The team achieved notable outcomes throughout the project, demonstrating strong integration of engineering, programming, and testing skills. Although constructing a fully scaled miniature road was beyond project feasibility, a practical testing environment with potholes marked directly on the ground was created to simulate real-world defect detection scenarios effectively. Using this setup, the robot autonomously navigated the area, leveraging its LiDAR-based SLAM mapping for localization while capturing high-resolution images of pothole markers. This data collection phase enabled the team to train and refine their AI-based pothole detection algorithm, improving its accuracy and reducing false positives through iterative calibration.

Furthermore, a real-time dashboard interface was developed, providing live visualisation of the robot's operations, including pothole detection markers overlaid on the video feed, severity classifications, route coverage maps, battery status, and system warnings. This integration demonstrated the entire envisioned operational pipeline – from navigation and data collection to processing, detection, and actionable reporting. The system also proved its robustness through multiple successful test runs, with the TurtleBot consistently detecting potholes and marking them accurately on the live dashboard without manual intervention, highlighting the effectiveness of sensor integration and autonomous control logic.

Additionally, the project offered valuable team learning outcomes, enhancing members' expertise in ROS programming, LiDAR mapping, machine learning implementation, and modular robotics integration – all critical competencies for modern robotics and smart infrastructure careers. The culmination of this comprehensive effort was recognised at the Innovation Fair awards ceremony, where the project earned 2nd place, validating not only its technical innovation but also its practical vision for improving road maintenance workflows. This achievement stands as a testament to the team's effective problem-solving, adaptability in overcoming environmental and hardware constraints, and commitment to developing solutions with genuine real-world impact.

12. Impact & Value Proposition

The PaveWay project carries significant impact potential by addressing a persistent urban infrastructure challenge through a practical, scalable, and technologically advanced solution. Roads worldwide deteriorate rapidly due to constant vehicular load and weather exposure, with potholes posing safety hazards, increasing vehicle maintenance costs, and disrupting traffic flow. Traditionally, inspections rely on manual surveys, which are inefficient, labour-intensive, and often delayed until public complaints arise. The PaveWay system reimagines this process by introducing autonomous, AI-powered inspection capable of continuously scanning road surfaces, identifying potholes in real time, and classifying their severity with consistent accuracy. This removes human bias, increases inspection frequency, and allows municipalities to prioritise repairs based on data-driven urgency, improving road safety and optimising limited maintenance budgets.

Technically, the project showcases the integration of LiDAR-based SLAM navigation with machine learning algorithms, enabling the robot to detect potholes accurately while autonomously navigating complex environments. The development of a live dashboard amplifies its value by providing maintenance teams with immediate, actionable insights, including pothole locations, severity markers, and inspection coverage maps, thus streamlining maintenance planning and execution. Although current tests were performed in a controlled environment with potholes marked on the ground rather than full-scale road deployment, the project successfully validated the core concept, proving its feasibility with minimal resources.

From an economic perspective, automating inspections reduces labour costs, minimises traffic disruption caused by manual survey crews, and prevents escalating repair expenses by detecting defects earlier. Societally, it enhances public safety by reducing accidents or vehicle damage associated with unnoticed potholes. Environmentally, prioritised, timely repairs can extend the life of road surfaces, reducing material waste and resource consumption associated with full road resurfacing.

Ultimately, combining Raspberry Pi processing, LiDAR mapping, and AI image analysis provides flexibility for future upgrades, such as integration with robotic swarms for city-wide coverage, or adaptation to detect additional road defects like cracks, faded lane markings, or drainage issues. Its successful demonstration and recognition as 2nd place at the Innovation Fair awards ceremony validate the project's practical potential and innovative approach, reflecting both technical excellence and real-world relevance.

It should be apparent that PaveWay represents a transformative leap towards smart, automated infrastructure management, paving the way for safer roads, optimised city budgets, and improved quality of life for all road users.

13. Costing & Financial Model

13.1 Development Cost (Prototype Phase)

Item	Estimated Cost (USD)
TurtleBot3 Burger Kit	\$550
Raspberry Pi 4 (4GB) + Accessories	\$100
Pi Camera 3	\$25
Misc. Hardware (mounts, wiring)	\$50
Training Dataset Collection	\$300

Software Development (Labor)	\$4,000
Testing & Simulation	\$500

Total Development Cost: ≈ \$5,525

13.2 Operational/Production Cost (Per Unit)

Item	Cost (USD)
TurtleBot3 Base (Bulk)	\$400
Compute Unit (RPI or SBC)	\$75
Camera	\$20
Battery & Charging Gear	\$60
Manufacturing & Assembly	\$100
Software Licensing/Support	\$45

Estimated Unit Cost: ≈ \$700 per robot

13.3 Business Model – Data-as-a-Service (DaaS)

- **Client:** Municipalities, city councils, civil engineering firms
- **Model:**
 - Deploy robots in cities to inspect roads regularly
 - Streamline pothole detection & location data to city dashboards
 - Offer subscription service per road segment or per sq. km

Tier	Price Estimate
Small city (10 km ²)	\$3,000/year
Medium (50 km ²)	\$12,000/year
Large (100+ km ²)	\$25,000/year

Revenue includes access to:

- Real-time map updates
- Historical reports & predictive maintenance data
- Integration with public works tools

13.4 Estimated Yearly Revenue

Assuming initial deployment in **10 small to medium municipalities**:

- Avg. Revenue per City: ~\$6,000/year
- Total Revenue (Year 1): **\$60,000**
- Growth Projection (Year 3): 30 clients × \$8,000 avg. = **\$240,000/year**

13.5 ROI Overview

Metric	Amount
Initial Investment	\$15,000 (5–7 robots + dev)
Yearly Operational Cost	~\$10,000
Year 1 Revenue	\$60,000
Net Profit (Year 1)	\$50,000
Payback Period	< 4 months

Break-even Point: Within the first **year of deployment**, assuming modest scaling and municipal interest.

14. Recommendations

As a prototype, PaveWay has successfully demonstrated the feasibility of using autonomous robotics and AI for pavement inspection. However, for the system to mature into a fully deployable municipal solution, a range of enhancements across hardware, software, user interaction, and operational frameworks must be considered. These recommendations aim to improve performance, extend functionality, and ensure long-term reliability, scalability, and adoption by city authorities.

The following section outlines strategic improvements categorized under three key domains: Technical Enhancements, User Experience Improvements, and Operational Deployment Strategies.

To transition PaveWay from prototype to full deployment, several improvements are recommended across technical, user, and operational domains. These focus areas ensure higher performance, system resilience, and long-term adoption by city or infrastructure authorities.

14.1 Technical Enhancements

- **Improved Sensing:** Add stereo/depth cameras for 3D crack profiling and infrared sensors for edge detection in low-light conditions.
- **AI Optimization:** Expand datasets for more robust model training and apply deep learning techniques (e.g., CNNs) to improve defect classification accuracy.
- **Edge Computing:** Upgrade to platforms like NVIDIA Jetson Nano for faster AI inference and reduced reliance on cloud resources.
- **Energy Efficiency:** Incorporate solar-assisted charging or modular battery systems to extend operation times in outdoor conditions.

14.2 Dashboard and Interface Enhancements

The current PaveWay dashboard provides real-time monitoring during inspections, including:

- **Live camera feed** with severity tagging.
- **Map tracking** of the robot's path.
- **Battery and inspection metrics** (time, distance, pothole count).
- **Manual controls** and **system alerts** for operational status.

To enhance usability and impact, future versions should include an analytics dashboard offering:

- **Severity breakdowns** and a Pothole Severity Index (1–5) to prioritize urgent repairs.
- **Monthly trends** in detection and repair for better maintenance planning.
- **Geographic distribution** of defects across regions to guide resource allocation.
- **Repair tracking** and image galleries for visual validation and progress reports.

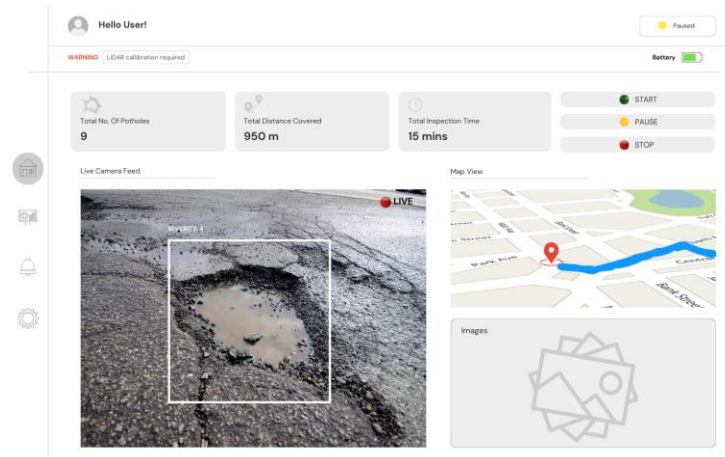


Figure 14.2.1: Dashboard (Home Page)

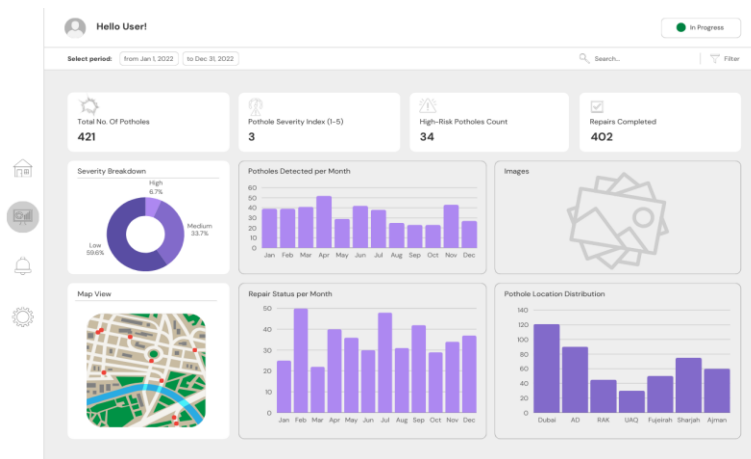


Figure 14.2.2: Dashboard (Analytics Page)

These features would turn the dashboard into a powerful decision-making tool for infrastructure authorities, supporting predictive maintenance and smarter road management.

14.3 Deployment Strategies

- **Pilot Deployments:** Start with controlled zones (campuses, public streets, commercial areas) to assess terrain adaptability and fine-tune performance.
- **Municipal System Integration:**
 - Link PaveWay data to city ERP and GIS systems.
 - Automate job order generation for detected defects.
- **Documentation & Training:**
 - Develop SOPs and maintenance manuals.
 - Provide user training for technical and non-technical staff.

14.4 Scalability & Innovation

- Prepare for multi-robot coordination with a centralized command interface.
- Future features may include thermal sensors or environmental monitoring modules.
- Explore networked solar charging stations to support 24/7 deployment.

15. Sustainability and SDG Alignment

Sustainability is at the heart of the PaveWay project's vision, influencing both its technical design and its broader societal purpose. As cities worldwide grapple with deteriorating infrastructure and rising environmental concerns, innovations like PaveWay offer a blueprint for transforming traditional maintenance practices into smarter, greener, and more inclusive systems.

The system aligns strongly with the United Nations Sustainable Development Goals (SDGs), particularly SDG 11: Sustainable Cities and Communities, by promoting responsible infrastructure upkeep, enabling data-driven urban planning, and reducing the environmental footprint of road maintenance.

15.1 Reducing Environmental Impact Through Smart Inspection

One of the key sustainable features of PaveWay lies in its hybrid deployment approach. For broader roads, the detection system can be mounted onto existing municipal vehicles, eliminating the need for separate inspection fleets and reducing additional fuel consumption. This allows road condition data to be collected seamlessly as part of routine vehicle movement. For pavements, bike lanes, and footpaths, areas not accessible by standard vehicles, the compact, energy-efficient robot takes over, conducting autonomous inspections without contributing to emissions or traffic. This dual approach ensures wider coverage with minimal environmental impact, supporting more sustainable and proactive infrastructure maintenance.

Additionally, the system encourages predictive maintenance, which plays a critical role in sustainability. When defects like potholes are detected early, they can be repaired with minimal material and labor. This contrasts with deferred maintenance, which often results in major repairs involving asphalt resurfacing, heavy equipment, and extended road closures, all of which significantly increase emissions and resource usage.

15.2 Social Sustainability and Workforce Safety

From a social sustainability standpoint, PaveWay addresses one of the most overlooked aspects of municipal operations: worker safety and labor equity. Manual inspections expose city workers to potential hazards, including heat stress, road accidents, and repetitive strain injuries. With PaveWay handling the routine, repetitive, and hazardous aspects of inspection, municipalities can redeploy workers to more skilled or supervisory roles, promoting healthier and safer working conditions.

Moreover, the data collected by PaveWay can enhance inclusive urban development. By objectively mapping road quality across various districts, including underserved neighborhoods, it ensures that infrastructure resources are distributed more equitably rather than being concentrated in more affluent areas with higher complaint volumes.

15.2.1 Economic Sustainability Through Preventive Maintenance

The economic benefits of the system are deeply tied to its ability to reduce long-term maintenance costs. When pavement issues are detected early, minor and inexpensive repairs can be scheduled, avoiding more extensive and costly reconstruction. This not only extends the service life of roads but also allows municipalities to optimize budget allocations, forecast maintenance needs, and prioritize repairs based on data rather than guesswork.

Furthermore, PaveWay promotes asset preservation, helping cities to avoid the costly consequences of infrastructure neglect. This is especially critical in developing regions where limited budgets make it essential to stretch the lifespan of existing assets.

15.2.2 Alignment with SDG 11: Sustainable Cities and Communities

The robot aligns with several specific targets under SDG 11:

- **Target 11.2:** Provide access to safe, affordable, accessible, and sustainable transport systems. PaveWay contributes by ensuring safer roads and walkways for all, including vulnerable populations such as the elderly and people with disabilities.
- **Target 11.6:** Reduce the adverse per capita environmental impact of cities. PaveWay minimizes environmental damage through efficient, low-power inspection and fewer emissions from heavy repair vehicles.
- **Target 11.3:** Enhance inclusive and sustainable urbanization. Its data-driven nature supports urban planners in identifying areas in need, ensuring fairer distribution of infrastructure services.

15.2.3 Long-Term Vision for Green Infrastructure Management

The PaveWay project is not just a standalone tool, it represents a shift toward digital, sustainable asset management systems. Its potential to integrate with smart city frameworks, GIS databases, and urban planning software means it can serve as a foundational component in a larger ecosystem of sustainable infrastructure technologies. Looking forward, enhancements such as solar charging, eco-friendly materials, or modular reuse of parts could further strengthen the sustainability profile of future PaveWay iterations.

16. Conclusion & Future Work

The PaveWay project successfully delivered a functional prototype for autonomous pavement and road inspection. By integrating SLAM-based navigation, AI-driven pothole detection, and real-time dashboard reporting, the system demonstrated its ability to automate a traditional manual and labor-intensive process.

Tested in realistic environments, the robot achieved accurate movement, reliable surface defect detection, and seamless data communication within a constrained budget. The project's modular design and iterative development enabled effective integration of mechanical, electrical, and software systems. Recognition at the Innovation Fair highlighted its innovation and real-world relevance. Future improvements include enhancing outdoor durability, extending battery life, integrating GPS, and expanding the detection system to include cracks and faded lane markings. Multi-robot coordination and predictive maintenance analytics can also be explored for large-scale deployments. PaveWay lays the groundwork for smarter, safer, and more efficient road maintenance, aligning with smart city goals and promoting sustainable urban infrastructure management.

17. References

[1] "ROBOTIS e-Manual," ROBOTIS e-Manual.

<https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/>

[2] ROS2 Documentation

<https://docs.ros.org/en/humble/index.html>

[3] ROS2 Cartographer

https://ros2-industrial-workshop.readthedocs.io/en/latest/_source/navigation/ROS2-Cartographer.html

[4] Ultralytics YOLOv8

<https://docs.ultralytics.com/>

[5] For Dataset - Roboflow

<https://roboflow.com/>

[6] Turtlebot3 Burger Repository

<https://github.com/ROBOTIS-GIT/turtlebot3>

18. Appendix

Link to GitHub Repository: [PaveWay Repository](#)

Gantt Chart

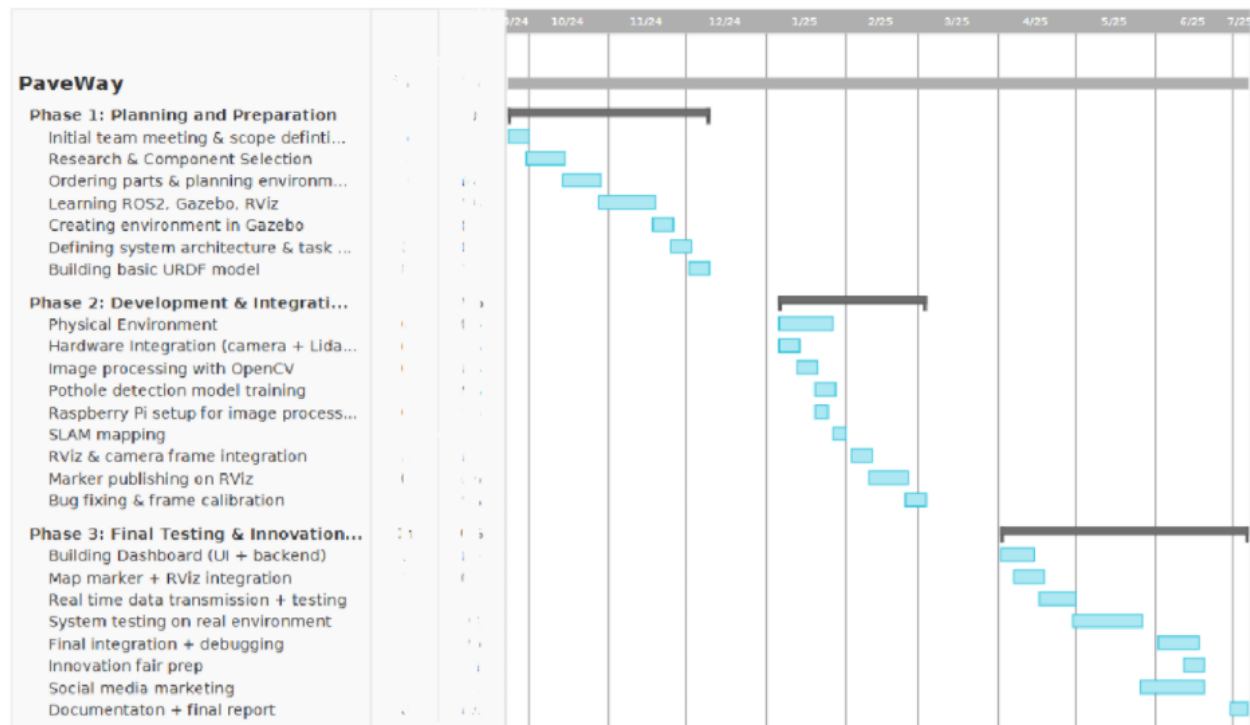


Figure 18.2: Gantt Chart



Figure 18.2: PaveWay Poster

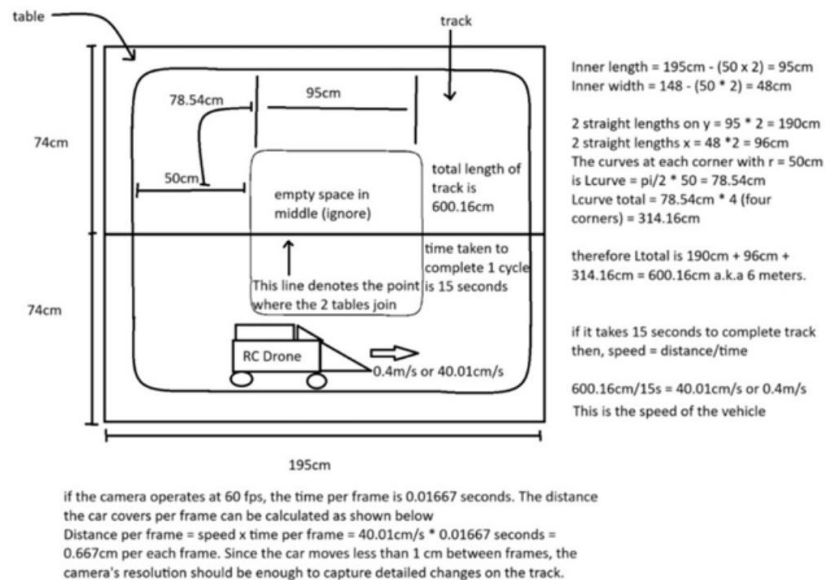


Figure 18.3: Track Measurements